

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 3 – Searches & sorts - **Solutions**

(1) Let G be an undirected graph with n vertices and m edges.

- (a) True or false: Every DFS forest for G will have the same number of trees, regardless of which vertex you start at.
- (b) True or false: Every DFS forest for G will have the same number of tree edges and the same number of back edges, regardless of which vertex you start at.

Justify each of your answers.

(a) True. Each connected component will produce one DFS tree.

(b) True. Each tree will have k vertices, where k is the number of vertices in that connected component. Since a tree is, by definition, minimally connected, all trees on k vertices have $k - 1$ edges, regardless of the topology, and the remaining (non-tree) edges will all be back edges.

(2) A graph is called *bipartite* if the vertices can be divided up into two groups, A and B , such that every edge in the graph connects a vertex from A and a vertex from B . (Put another way, there are no *internal* edges within A , and also none within B .)

- (a) Give an algorithm based upon breadth-first search for determining whether or not an undirected graph is bipartite.
- (b) What will be true of the cross edges in the BFS tree of a connected, bipartite graph? Explain why this is the case.

(a) Label the root of the BFS tree as in group A . Then all of its neighbors must be in group B . These neighbors are precisely the nodes in the first level of the BFS tree, and we will label them with a B . Their neighbors must be in A ; this is the second level of the BFS tree, and so on, with the layers of the tree alternating between A and B . If at any point, we encounter a cross edge that connects to a node with same label, the graph is not bipartite. If we get through a full BFS without this happening, the graph is bipartite. (The nice thing about this is that it is a *constructive* solution – when we're done, we not only know it's bipartite but we know how to divide the nodes up.)

(b) Cross edges in a BFS tree are always between nodes in the same layer, or in nodes that differ by at most one layer. (Why?) Bipartite graphs cannot have cross edges between nodes of the same layer; thus, all cross edges will connect nodes that are precisely one layer apart.

(3) Demonstrate the execution of insertion sort on the letters of the word PROSECUTIONS. You should show the new order after each time a letter is moved. Also, use subscripts to distinguish multiple copies of the same letter. (i.e. Your initial positions should be written as $PRO_1S_1ECUTIO_2NS_2$.) Ensure that your sort is stable.

(Note to TAs: Students do not have to show the “no movement” steps.)

$PRO_1S_1ECUTIO_2NS_2$
 $PRO_1S_1ECUTIO_2NS_2$ (no movement)
 $O_1PRS_1ECUTIO_2NS_2$
 $O_1PRS_1ECUTIO_2NS_2$ (no movement)
 $EO_1PRS_1CUTIO_2NS_2$
 $CEO_1PRS_1UTIO_2NS_2$
 $CEO_1PRS_1UTIO_2NS_2$ (no movement)
 $CEO_1PRS_1TUIO_2NS_2$
 $CEIO_1PRS_1TUIO_2NS_2$
 $CEIO_1O_2PRS_1TUNS_2$
 $CEINO_1O_2PRS_1TUS_2$
 $CEINO_1O_2PRS_1S_2TU$

(4) (a) Consider the operation of insertion sort on a singly-linked list. Which of the “nice” properties of insertion sort will be lost. Explain why.

(b) Consider the operation of insertion sort on a doubly-linked list. Will it lose any of the “nice” properties? Why or why not?

(a) We cannot easily walk “backwards” down the list. If we stick with the standard ordering of insertion sort, it will no longer be adaptive, nor particularly fast on small cases.

It is possible to be a bit more clever about it by working from the end of the list backward using a recursive approach. This can save both of these properties, but at the cost of using a linear amount of memory on the stack, so it would no longer be considered in-place.

(b) A doubly-linked list has no issues – we can traverse the list in both directions, and swapping elements around just involves moving some pointers, so the standard insertion sort implementation works fine. (Note, however, that an extra pointer for each element is an $O(n)$ amount of additional memory. If you’re already storing that pointer, no big deal. But if you need to add it solely so that your insertion sort works well, once again it would be disingenuous to call it “in-place”.)

(5) Give a non-recursive algorithm for generating the binary reflected Gray code of length n . (Hint: The BRGC of length n is very strongly related to the moves in a Towers of Hanoi puzzle of size n .) What is the runtime of your algorithm?

Multiple approaches possible; one is given here. All reasonable algorithms should be $\Theta(2^n)$.

The trick is that bit 0 flips on turns 1,3,5,...; bit 1 flips on turns 2,6,10,...; bit 2 flips on turns 4,12,20,..., and in general, bit k flips on turns $2^k, 3 \cdot 2^k, 5 \cdot 2^k, \dots$

So the algorithm for n bits goes like this:

- Initialize code to 000...000 (n zeroes, with bits labeled b_{n-1} to b_0), and print it.
- For $i = 1$ to $2^n - 1$
 - Toggle b_k , where k is the largest integer for which $i \div 2^k$ has no remainder
 - Print the code.

(6) An array of size $n - 1$ contains one copy each of every integer from 1 to n in ascending order, leaving out a single unknown integer. Design the most efficient algorithm you can for determining which integer has been left out. What is the runtime of your algorithm?

Many possible answers here. Since the list is sorted, the “default” approach is simply to scan through it until you find the missing item, which is $O(n)$; nothing slower than this should be accepted.

Another possibility is using Gauss’s closed-form formula for $\sum_{i=1}^n i$, adding up each element in the list, and then taking the difference. This is also linear time.

The best approaches will involve a modified binary search. At each stage, if the value is at its correct index, the missing item must be higher up; if the value is off by 1, the missing item is either here or lower down. This is the optimal approach; it runs in $O(\log n)$ time.